

ФИНКИ

Факултет за Информатички Науки и Компјутерско Инженерство

КИИИ - Континуирана Интеграција и Испорака

Containerization, CI Pipeline and Orchestration of a Full-Stack To-Do Application

Студент: Стефан Тасевски

Индекс: 233105

Професори: проф. Панче Рибарски

Асистент: Стефан Андонов

Семестар 6

12 Август 2026

1. Преглед на Проектот и Архитектура

1.1 Опис на Проектот

Проектот демонстрира имплементација на DevOps решение на едноставна full-stack апликација со React frontend, Node.js/Express backend и PostgreSQL база на податоци. Апликацијата е едноставна CRUD апликација за управување со листа од задачи, која служи како демонстрација на DevOps практиките вклучувајќи контејнеризација, оркестрација, CI пајплајн и Kubernetes оркестрација.

1.2 Технолошка Ставка

- Frontend: React 18.3.1 со Vite 6.4.3 за брз развој и градба
- Backend: Node.js 24 со Express.js 4.21.0 за REST API-ја
- База на Податоци: PostgreSQL 16-Alpine за релациона датотека
- Контејнеризација: Docker со Docker compose
- Оркестрации: Docker compose и Kubernetes преку k3d
- CI/CD Платформа: GitHub Actions за автоматизирана градба и push на images на Docker Hub

1.3 Архитектура на Апликацијата

Апликацијата е организирана во три главни компоненти:

Frontend: React апликација која работи на порт 80 во контејнер, сервирана преку Nginx. Комуницира со backend API преку HTTP.

Backend: Node.js Express сервер кој работи на порт 3001. Изложува RESTful API за управување со задачи и здравствено проверување (health check).

Датотека: PostgreSQL база која чува информации за задачите. Зачувана во Docker volume за персистентност на податоците.

2. Детали на Имплементацијата

2.1 Контејнеризација со Docker

Секоја компонента на апликацијата е контејнеризирана:

- **Frontend:** Multi-stage Dockerfile кој прво градува React апликација со npm, потоа сервира static files преку Nginx
- **Backend:** Dockerfile кој користи Node 24-Alpine базна слика, инсталира зависности и стартува Node сервер
- **База на Податоци:** PostgreSQL 16-Alpine официјална слика од Docker Hub

2.2 Docker Compose Оркестрација

`docker-compose.yml` дефинира три сервиси (frontend, backend, db). Зависностите се дефинирани така што backend чека на базата пред стартување, а frontend чека на backend. Волумен db-data е дефиниран за персистентност на PostgreSQL податоците.

2.3 GitHub Actions CI/CD Пајплајн

Пајплајнот е конфигуриран да се активира на push кон master гранката. Истиот се состои од следното:

1. **Checkout:** Превземање на кодот од репозиториумот
2. **Setup Node.js 24:** Инсталирање на Node.js за градба
3. **Градба на Frontend:** npm install и npm run build
4. **Градба на Backend:** npm install --omit=dev
5. **Docker Login:** Пријава на Docker Hub користејќи GitHub Secrets
6. **Build и Push Frontend:** Градба и push на фронтенд слика на Docker Hub
7. **Build и Push Backend:** Градба и push на бекенд слика на Docker Hub

2.4 Kubernetes Деплојмент

Kubernetes кластерот е изграден од елементи кои се дефинирани во манифестите во k8s/ папката. Преку истите, кластерот содржи:

- **Namespace:** „dev-ops-project“ за изолација на ресурсите
- **ConfigMap:** За нетајни конфигурациски вредности (DB_HOST, DB_PORT, итн.)
- **Secret:** За тајни податоци како DB_PASSWORD
- **PostgreSQL StatefulSet:** За базата со persistent storage+
- **Backend Deployment:** За Node.js сервер со livenessProbe и readinessProbe здравствени проверки
- **Frontend Deployment:** За React апликација сервирана преку Nginx
- **Service:** За изложување на frontend и backend во кластерот
- **Ingress:** За рутирање на надворешен трафик преку Traefik контролер

3. Деплојмент и Резултати

3.1 Тестирање на Docker Compose

За тестирање на docker композицијата, на root директориумот се стартува со:

```
docker compose up --build
```

Ова стартува сите три компоненти (frontend, backend, database) во Docker контејнери. Апликацијата е достапна на **http://localhost:3000** за frontend и **http://localhost:3001** за backend API.

3.2 Тестирање на Кластерот

За локално тестирање со k3d прво се креира кластер користејќи:

```
k3d cluster create devops-project-cluster -p "3000:80@loadbalancer" -p  
"3001:80@loadbalancer" -s 1 -a 1
```

Потоа се извршуваат следните команди за да се применат манифестите:

- **kubectl apply -f k8s/namespace.yaml**
- **kubectl apply -f k8s/configmap.yaml**
- **kubectl apply -f k8s/secret.yaml**
- **kubectl apply -f k8s/postgres-statefulset.yaml**
- **kubectl apply -f k8s/backend-deployment.yaml**
- **kubectl apply -f k8s/frontend-deployment.yaml**
- **kubectl apply -f k8s/frontend-ingress.yaml**
- **kubectl apply -f k8s/backend-ingress.yaml**

Од тука, се користи командата **kubectl get ingress -n dev-ops-project** – при што обрнуваме внимание на “Adresses” колоната од генерираната табела. Се додека има непостоечки вредности мораме да почекаме да се иницијализираат истите. Повторно се применува истата команда за проверка се додека не се појават вредностите.

Штом се иницијализираат адресите, го применуваме последниот манифест:

```
kubectl apply -f k8s/backend-middleware.yaml
```

Следно, во widnows hosts фајлот го додаваме 127.0.0.1 devops-project.local со цел да може да се резолвира DNS името кога ќе го повикаме browser.

Со тоа, кластерот целосно функционира.

3.4 Верификација на Kubernetes cluster

За да се осигураме дека навистина кластерот правилно работи, ги користиме следните команди:

- Проверување на подови: **kubectl get pods -n dev-ops-project**
- Проверување на сервиси: **kubectl get svc -n dev-ops-project**
- Проверување на Ingress: **kubectl get ingress -n dev-ops-project**
- Пристап кон апликацијата - На <http://devops-project.local:3000> е достапен frontend-от, а на <http://localhost:3001/health> можеме да направиме health check на backend-от.

3.5 Клучни Достигнувања

- ✓ Целосна контејнеризација на full-stack апликација
- ✓ Успешна оркестрација со Docker Compose за развој
- ✓ Автоматизиран CI пајплајн преку GitHub Actions
- ✓ Автоматизирана градба и push на docker image на Docker Hub при push на git репозиториумот
- ✓ Kubernetes манифести за production деплојмент
- ✓ Здравствени проверки (readiness и liveness) за самовосстановување
- ✓ Безбедни конфигурации преку ConfigMaps и Secrets
- ✓ Ingress контролер за управување со сообраќај кон и надвор од кластерот
- ✓ Persistent storage за PostgreSQL база на податоци

3.6 Заклучок

Овој проект демонстрира примена на бројни DevOps методолошки и технологии. Од контејнеризација преку Docker, автоматизирана интеграција преку GitHub Actions, до оркестрирање и управување со Kubernetes, проектот покрива суштински DevOps вештини потребни за модерно развивање и деплојмент на софтверски апликации. Скалибилноста, доверливоста и автоматизацијата постигнати преку овој проект служат како темелна основа за production-grade системи.

4. Линкови

4.1 Github репозиториум

Проектот е достапен на јавниот git репозиториум:

<https://github.com/TaseZmaj/Continuous-Integration-and-Continuous-Delivery-Project>